A slightly longer Lean 4 proof tour

Terence Tao · 5 December 2023

https://terrytao.wordpress.com/2023/12/05/a-slightly-longer-lean-4-proof-tour/

tao2typ · converted to Typst 2026-10-14 (UTC)

In my previous post, I walked through the task of formally deducing one lemma from another in Lean 4. The deduction was deliberately chosen to be short and only showcased a small number of Lean tactics. Here I would like to walk through the process I used for a slightly longer proof I worked out recently, after seeing the following challenge from Damek Davis: to formalize (in a civilized fashion) the proof of the following lemma:

> **Lemma**. Let $\{a_k\}$ and $\{D_k\}$ be sequences of real numbers indexed by natural numbers $k = 0, 1, \ldots$, with a_k non-increasing and D_k non-negative. Suppose also that $a_k \leq D_k - D_{k+1}$ for all $k \geq 0$. Then $a_k \leq \frac{D_0}{k+1}$ for all k.

Here I tried to draw upon the lessons I had learned from the PFR formalization project, and to first set up a human readable proof of the lemma before starting the Lean formalization – a lower-case "blueprint" rather than the fancier Blueprint used in the PFR project. The main idea of the proof here is to use the telescoping series identity

$$\sum_{i=0}^{k} D_i - D_{i+1} = D_0 - D_{k+1}.$$

Since D_{k+1} is non-negative, and $a_i \leq D_i - D_{i+1}$ by hypothesis, we have

$$\sum_{i=0}^{k} a_i \leq D_0$$

but by the monotone hypothesis on a_i the left-hand side is at least $(k+1)a_k$, giving the claim.

This is already a human-readable proof, but in order to formalize it more easily in Lean, I decided to rewrite it as a chain of inequalities, starting at a_k and ending at $D_0/(k+1)$. With a little bit of pen and paper effort, I obtained

$a_k = (k+1)a_k/(k+1)$
(by field identities)
$= (\sum_{i=0}^{k} a_k)/(k+1)$
(by the formula for summing a constant)
$\leq (\sum_{i=0}^{k} a_i)/(k+1)$
(by the monotone hypothesis)
$\leq (\sum_{i=0}^{k} D_i - D_{i+1})/(k+1)$
(by the hypothesis $a_i \leq D_i - D_{i+1}$
$= (D_0 - D_{k+1})/(k+1)$
(by telescoping series)
$\leq D_0/(k+1)$
(by the non-negativity of D_{k+1}).

I decided that this was a good enough blueprint for me to work with. The next step is to formalize the statement of the lemma in Lean. For this quick project, it was convenient to use the online Lean playground, rather than my local IDE, so the screenshots will look a little different from those in the previous post. (If you like, you can follow this tour in that playground, by clicking on the screenshots of the Lean code.) I start by importing Lean's math library, and starting an example of a statement to state and prove:

```
1 import Mathlib
2
3 example
```

```
1 import Mathlib
2
3 example
```

Now we have to declare the hypotheses and variables. The main variables here are the sequences a_k and D_k, which in Lean are best modeled by functions `a`, `D` from the natural numbers $\mathbb{N}$ to the reals $\mathbb{R}$. (One can choose to "hardwire" the non-negativity hypothesis into the D_k by making `D` take values in the nonnegative reals $\mathbf{R}^+$ (denoted `NNReal` in Lean), but this turns out to be inconvenient, because the laws of algebra and summation that we will need are clunkier on the non-negative reals (which are not even a group) than on the reals (which are a field). So we add in the variables:

```
1 import Mathlib
2
3 example (a D : ℕ → ℝ)
```

Now we add in the hypotheses, which in Lean convention are usually given names starting with `h`. This is fairly straightforward; the one thing is that the property of being monotone decreasing already has a

name in Lean's Mathlib, namely `Antitone`, and it is generally a good idea to use the Mathlib provided terminology (because that library contains a lot of useful lemmas about such terms).

```
import Mathlib

example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a)
```

One thing to note here is that Lean is quite good at filling in implied ranges of variables. Because `a` and `D` have the natural numbers ℕ as their domain, the dummy variable `k` in these hypotheses is automatically being quantified over ℕ. We *could* have made this quantification explicit if we so chose, for instance using `∀ k : ℕ, 0 ≤ D k` instead of `∀ k, 0 ≤ D k`, but it is not necessary to do so. Also note that Lean does not require parentheses when applying functions: we write `D k` here rather than `D(k)` (which in fact does not compile in Lean unless one puts a space between the `D` and the parentheses). This is slightly different from standard mathematical notation, but is not too difficult to get used to.

This looks like the end of the hypotheses, so we could now add a colon to move to the conclusion, and then add that conclusion:

```
import Mathlib

example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a)
  : ∀ k, a k ≤ D 0 / (k + 1)
```

This is a perfectly fine Lean statement. But it turns out that when proving a universally quantified statement such as `∀ k, a k ≤ D 0 / (k + 1)`, the first step is almost always to open up the quantifier to introduce the variable `k` (using the Lean command `intro k`). Because of this, it is slightly more efficient to hide the universal quantifier by placing the variable `k` in the hypotheses, rather than in the quantifier (in which case we have to now specify that it is a natural number, as Lean can no longer deduce this from context):

```
import Mathlib

example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
  : a k ≤ D 0 / (k + 1)
```

At this point Lean is complaining of an unexpected end of input: the example has been stated, but not proved. We will temporarily mollify Lean by adding a `sorry` as the purported proof:

```
import Mathlib

example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
  : a k ≤ D 0 / (k + 1) := sorry
```

Now Lean is content, other than giving a warning (as indicated by the yellow squiggle under the `example`) that the proof contains a sorry.

It is now time to follow the blueprint. The Lean tactic for proving an inequality via chains of other inequalities is known as `calc`. We use the blueprint to fill in the `calc` that we want, leaving the justifications of each step as "`sorry`"s for now:

```
import Mathlib

open Finset BigOperators

example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
  : a k ≤ D 0 / (k + 1) := calc
    a k = (k+1) * (a k) / (k+1)                     := by sorry
    _ = (∑ i in range (k+1), a k) / (k+1)           := by sorry
    _ ≤ (∑ i in range (k+1), a i) / (k+1)           := by sorry
    _ ≤ (∑ i in range (k+1), (D i - D (i+1))) / (k+1) := by sorry
    _ = (D 0 - D (k+1)) / (k+1)                     := by sorry
    _ ≤ D 0 / (k+1)                                 := by sorry
```

Here, we "`open`"ed the `Finset` namespace in order to easily access `Finset`'s `range` function, with `range k` basically being the finite set of natural numbers $\{0, \dots, k-1\}$, and also "`open`"ed the `BigOperators` namespace to access the familiar Σ notation for (finite) summation, in order to make the steps in the Lean code resemble the blueprint as much as possible. One could avoid opening these namespaces, but then expressions such as `∑ i in range (k+1), a i` would instead have to be written as something like `Finset.sum (Finset.range (k+1)) (fun i ↦ a i)`, which looks a lot less like like standard mathematical writing. The proof structure here may remind some readers of the "two column proofs" that are somewhat popular in American high school geometry classes.

Now we have six sorries to fill. Navigating to the first sorry, Lean tells us the ambient hypotheses, and the goal that we need to prove to fill that sorry:

```
a D : ℕ → ℝ
hD : ∀ (k : ℕ), a k ≤ D k - D (k + 1)
hpos : ∀ (k : ℕ), 0 ≤ D k
ha : Antitone a
k : ℕ
⊢ a k = (↑k + 1) * a k / (↑k + 1)
```

The $\vdash$ symbol here is Lean's marker for the goal. The uparrows $\uparrow$ are coercion symbols, indicating that the natural number `k` has to be converted to a real number in order to interact via arithmetic operations with other real numbers such as `a k`, but we can ignore these coercions for this tour (for this proof, it turns out Lean will basically manage them automatically without need for any explicit intervention by a human).

The goal here is a self-evident algebraic identity; it involves division, so one has to check that the denominator is non-zero, but this is self-evident. In Lean, a convenient way to establish algebraic identities is to use the tactic `field_simp` to clear denominators, and then `ring` to verify any identity that is valid for commutative rings. This works, and clears the first `sorry`:

```
import Mathlib

open Finset BigOperators

example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
  : a k ≤ D 0 / (k + 1) := calc
    a k = (k+1) * (a k) / (k+1)                     := by field_simp; ring
    _ = (∑ i in range (k+1), a k) / (k+1)           := by sorry
    _ ≤ (∑ i in range (k+1), a i) / (k+1)           := by sorry
    _ ≤ (∑ i in range (k+1), (D i - D (i+1))) / (k+1) := by sorry
    _ = (D 0 - D (k+1)) / (k+1)                     := by sorry
    _ ≤ D 0 / (k+1)                                 := by sorry
```

`field_simp`, by the way, is smart enough to deduce on its own that the denominator `k+1` here is manifestly non-zero (and in fact positive); no human intervention is required to point this out. Similarly for other "clearing denominator" steps that we will encounter in the other parts of the proof.

Now we navigate to the next `sorry`. Lean tells us the hypotheses and goals:

```
a D : ℕ → ℝ
hD : ∀ (k : ℕ), a k ≤ D k - D (k + 1)
hpos : ∀ (k : ℕ), 0 ≤ D k
ha : Antitone a
k : ℕ
⊢ (↑k + 1) * a k / (↑k + 1) = (∑ i in range
(k + 1), a k) / (↑k + 1)
```

We can reduce the goal by canceling out the common denominator `↑k+1`. Here we can use the handy Lean tactic `congr`, which tries to match two sides of an equality goal as much as possible, and leave any remaining discrepancies between the two sides as further goals to be proven. Applying `congr`, the goal reduces to

```
a D : ℕ → ℝ
hD : ∀ (k : ℕ), a k ≤ D k - D (k + 1)
hpos : ∀ (k : ℕ), 0 ≤ D k
ha : Antitone a
k : ℕ
⊢ (↑k + 1) * a k = ∑ i in range (k + 1), a k
```

Here one might imagine that this is something that one can prove by induction. But this particular sort of identity – summing a constant over a finite set – is already covered by Mathlib. Indeed, searching for `Finset`, `sum`, and `const` soon leads us to the `Finset.sum_const` lemma here. But there is an even more convenient path to take here, which is to apply the powerful tactic `simp`, which tries to simplify the goal as much as possible using all the "`simp` lemmas" Mathlib has to offer (of which `Finset.sum_const` is an example, but there are thousands of others). As it turns out, `simp` completely kills off this identity, without any further human intervention:

```
1  import Mathlib
2
3  open Finset BigOperators
4
5  example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
6    : a k ≤ D 0 / (k + 1) := calc
7      a k = (k+1) * (a k) / (k+1)                    := by field_simp; ring
8      _ = (∑ i in range (k+1), a k) / (k+1)          := by congr; simp
9      _ ≤ (∑ i in range (k+1), a i) / (k+1)          := by sorry
10     _ ≤ (∑ i in range (k+1), (D i - D (i+1))) / (k+1) := by sorry
11     _ = (D 0 - D (k+1)) / (k+1)                    := by sorry
12     _ ≤ D 0 / (k+1)                                := by sorry
```

Now we move on to the next sorry, and look at our goal:

```
a D : ℕ → ℝ
hD : ∀ (k : ℕ), a k ≤ D k - D (k + 1)
hpos : ∀ (k : ℕ), 0 ≤ D k
ha : Antitone a
k : ℕ
⊢ (∑ i in range (k + 1), a k) / (↑k + 1) ≤
(∑ i in range (k + 1), a i) / (↑k + 1)
```

`congr` doesn't work here because we have an inequality instead of an equality, but there is a powerful relative `gcongr` of `congr` that is perfectly suited for inequalities. It can also open up sums, products, and integrals, reducing global inequalities between such quantities into pointwise inequalities. If we invoke `gcongr with i hi` (where we tell `gcongr` to use `i` for the variable opened up, and `hi` for the constraint this variable will satisfy), we arrive at a greatly simplified goal (and a new ambient variable and hypothesis):

```
a D : ℕ → ℝ
hD : ∀ (k : ℕ), a k ≤ D k - D (k + 1)
hpos : ∀ (k : ℕ), 0 ≤ D k
ha : Antitone a
k i : ℕ
hi : i ∈ range (k + 1)
⊢ a k ≤ a i
```

Now we need to use the monotonicity hypothesis on `a`, which we have named `ha` here. Looking at the documentation for Antitone, one finds a lemma that looks applicable here:

```
theorem Antitone.imp                                                source
      {α : Type u} {β : Type v} [Preorder α] [Preorder β]
      {f : α → β} {a : α} {b : α} (hf : Antitone f) (h : a ≤ b) :
   f b ≤ f a
```

One can apply this lemma in this case by writing `apply Antitone.imp ha`, but because `ha` is already of type `Antitone`, we can abbreviate this to `apply ha.imp`. (Actually, as indicated in the documentation, due to the way `Antitone` is defined, we can even just use `apply ha` here.) This reduces the goal nicely:

```
a D : ℕ → ℝ
hD : ∀ (k : ℕ), a k ≤ D k - D (k + 1)
hpos : ∀ (k : ℕ), 0 ≤ D k
ha : Antitone a
k i : ℕ
hi : i ∈ range (k + 1)
⊢ i ≤ k
```

The goal is now very close to the hypothesis `hi`. One could now look up the documentation for `Finset.range` to see how to unpack `hi`, but as before `simp` can do this for us. Invoking `simp at hi`, we obtain

```
a D : ℕ → ℝ
hD : ∀ (k : ℕ), a k ≤ D k - D (k + 1)
hpos : ∀ (k : ℕ), 0 ≤ D k
ha : Antitone a
k i : ℕ
hi : i < k + 1
⊢ i ≤ k
```

Now the goal and hypothesis are very close indeed. Here we can just close the goal using the `linarith` tactic used in the previous tour:

```
1 import Mathlib
2
3 open Finset BigOperators
4
5 example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
6   : a k ≤ D 0 / (k + 1) := calc
7     a k = (k+1) * (a k) / (k+1)                    := by field_simp; ring
8     _ = (∑ i in range (k+1), a k) / (k+1)          := by congr; simp
9     _ ≤ (∑ i in range (k+1), a i) / (k+1)          := by gcongr with i hi; apply ha; simp at hi; linarith
10    _ ≤ (∑ i in range (k+1), (D i - D (i+1))) / (k+1) := by sorry
11    _ = (D 0 - D (k+1)) / (k+1)                    := by sorry
12    _ ≤ D 0 / (k+1)                                := by sorry
```

The next sorry can be resolved by similar methods, using the hypothesis `hD` applied at the variable `i`:

```
1 import Mathlib
2
3 open Finset BigOperators
4
5 example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
6   : a k ≤ D 0 / (k + 1) := calc
7     a k = (k+1) * (a k) / (k+1)                    := by field_simp; ring
8     _ = (∑ i in range (k+1), a k) / (k+1)          := by congr; simp
9     _ ≤ (∑ i in range (k+1), a i) / (k+1)          := by gcongr with i hi; apply ha; simp at hi; linarith
10    _ ≤ (∑ i in range (k+1), (D i - D (i+1))) / (k+1) := by gcongr with i hi; exact hD i
11    _ = (D 0 - D (k+1)) / (k+1)                    := by sorry
12    _ ≤ D 0 / (k+1)                                := by sorry
```

Now for the penultimate sorry. As in a previous step, we can use `congr` to remove the denominator, leaving us in this state:

```
a D : ℕ → ℝ
hD : ∀ (k : ℕ), a k ≤ D k - D (k + 1)
hpos : ∀ (k : ℕ), 0 ≤ D k
ha : Antitone a
k : ℕ
⊢ ∑ i in range (k + 1), (D i - D (i + 1)) =
D 0 - D (k + 1)
```

This is a telescoping series identity. One could try to prove it by induction, or one could try to see if this identity is already in Mathlib. Searching for `Finset`, `sum`, and `sub` will locate the right tool (as the fifth hit), but a simpler way to proceed here is to use the `exact?` tactic we saw in the previous tour:

▼ Suggestion

```
Try this: exact
      sum_range_sub' (fun i => D i) (k + 1)
```

A brief check of the documentation for `sum_range_sub'` confirms that this is what we want. Actually we can just use `apply sum_range_sub'` here, as the `apply` tactic is smart enough to fill in the missing arguments:

```
1  import Mathlib
2
3  open Finset BigOperators
4
5  example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
6    : a k ≤ D 0 / (k + 1) := calc
7      a k = (k+1) * (a k) / (k+1)                    := by field_simp; ring
8      _ = (∑ i in range (k+1), a k) / (k+1)          := by congr; simp
9      _ ≤ (∑ i in range (k+1), a i) / (k+1)          := by gcongr with i hi; apply ha; simp at hi; linarith
10     _ ≤ (∑ i in range (k+1), (D i - D (i+1))) / (k+1) := by gcongr with i hi; exact hD i
11     _ = (D 0 - D (k+1)) / (k+1)                    := by congr; apply sum_range_sub'
12     _ ≤ D 0 / (k+1)                                := by sorry
```

One last `sorry` to go. As before, we use `gcongr` to cancel denominators, leaving us with

```
a D : ℕ → ℝ
hD : ∀ (k : ℕ), a k ≤ D k - D (k + 1)
hpos : ∀ (k : ℕ), 0 ≤ D k
ha : Antitone a
k : ℕ
⊢ D 0 - D (k + 1) ≤ D 0
```

This looks easy, because the hypothesis `hpos` will tell us that `D (k+1)` is nonnegative; specifically, the instance `hpos (k+1)` of that hypothesis will state exactly this. The `linarith` tactic will then resolve this goal once it is told about this particular instance:

```
1  import Mathlib
2
3  open Finset BigOperators
4
5  example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
6    : a k ≤ D 0 / (k + 1) := calc
7      a k = (k+1) * (a k) / (k+1)                    := by field_simp; ring
8      _ = (∑ i in range (k+1), a k) / (k+1)          := by congr; simp
9      _ ≤ (∑ i in range (k+1), a i) / (k+1)          := by gcongr with i hi; apply ha; simp at hi; linarith
10     _ ≤ (∑ i in range (k+1), (D i - D (i+1))) / (k+1) := by gcongr with i hi; exact hD i
11     _ = (D 0 - D (k+1)) / (k+1)                    := by congr; apply sum_range_sub'
12     _ ≤ D 0 / (k+1)                                := by gcongr; linarith [hpos (k+1)]
```

We now have a complete proof – no more yellow squiggly line in the example. There are two warnings though – there are two variables `i` and `hi` introduced in the proof that Lean's "linter" has noticed are not actually used in the proof. So we can rename them with underscores to tell Lean that we are okay with them not being used:

```
import Mathlib

open Finset BigOperators

example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
  : a k ≤ D 0 / (k + 1) := calc
    a k = (k+1) * (a k) / (k+1)                     := by field_simp; ring
    _ = (∑ _i in range (k+1), a k) / (k+1)          := by congr; simp
    _ ≤ (∑ i in range (k+1), a i) / (k+1)           := by gcongr with i hi; apply ha; simp at hi; linarith
    _ ≤ (∑ i in range (k+1), (D i - D (i+1))) / (k+1) := by gcongr with i _; exact hD i
    _ = (D 0 - D (k+1)) / (k+1)                     := by congr; apply sum_range_sub'
    _ ≤ D 0 / (k+1)                                 := by gcongr; linarith [hpos (k+1)]
```

This is a perfectly fine proof, but upon noticing that many of the steps are similar to each other, one can do a bit of "code golf" as in the previous tour to compactify the proof a bit:

```
import Mathlib

open Finset BigOperators

example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k - D (k+1)) (hpos: ∀ k, 0 ≤ D k) (ha : Antitone a) (k : ℕ)
  : a k ≤ D 0 / (k + 1) := calc
    a k = (∑ _i in range (k+1), a k) / (k+1) := by simp; field_simp; ring
    _ ≤ (∑ i in range (k+1), (D i - D (i+1))) / (k+1) := by
      gcongr with i hi; apply le_trans _ (hD i); apply ha; simp at hi; linarith
    _ ≤ D 0 / (k+1) := by gcongr; rw [sum_range_sub']; linarith [hpos (k+1)]
```

With enough familiarity with the Lean language, this proof actually tracks quite closely with (an optimized version of) the human blueprint.

This concludes the tour of a lengthier Lean proving exercise. I am finding the pre-planning step of the proof (using an informal "blueprint" to break the proof down into extremely granular pieces) to make the formalization process significantly easier than in the past (when I often adopted a sequential process of writing one line of code at a time without first sketching out a skeleton of the argument). (The proof here took only about 15 minutes to create initially, although for this blog post I had to recreate it with screenshots and supporting links, which took significantly more time.) I believe that a realistic near-term goal for AI is to be able to fill in automatically a significant fraction of the sorts of atomic "`sorry`"s of the size one saw in this proof, allowing one to convert a blueprint to a formal Lean proof even more rapidly.

One final remark: in this tour I filled in the "`sorry`"s in the order in which they appeared, but there is actually no requirement that one does this, and once one has used a blueprint to atomize a proof into self-contained smaller pieces, one can fill them in in any order. Importantly for a group project, these micro-tasks can be parallelized, with different contributors claiming whichever "`sorry`" they feel they are qualified to solve, and working independently of each other. (And, because Lean can automatically verify if their proof is correct, there is no need to have a pre-existing bond of trust with these contributors in order to accept their contributions.) Furthermore, because the specification of a "`sorry`" someone can make a meaningful contribution to the proof by working on an extremely localized component of it without needing the mathematical expertise to understand the global argument. This is not particularly important in this simple case, where the entire lemma is not too hard to understand to a trained mathematician, but can become quite relevant for complex formalization projects.

Comments (42)



wdjoyner · 6 December 2023

Thank you for the interesting post but I think there's a typo somewhere. After "I start by importing Lean's math library, and starting an example of a statement to state and prove:" the display is blank.
[Added, thanks – T.]

Anonymous · 6 December 2023

Re: the twitter challenge from Damek Davis.
“You’re unable to view this Post because this account owner limits who can view their Posts. Learn more”



Dustin G. Mixon · 6 December 2023

In the second display, the sum of a_i should be leq D_0.
[Corrected, thanks – T.]

Anonymous · 6 December 2023

Isn’t it Sum a_i <= D_0 rather than =

Anonymous · 7 December 2023

All rhs have / (k + 1) in common. The proof would be simpler if we prove (k+1)a_k \le D_0 first.



Terence Tao · 7 December 2023

This is certainly an option. Here is one arrangement of the proof that follows that strategy; there is one “rewrite” inserted at the beginning, and an number of `congr` and `gcongr` steps that were used to remove the denominator are removed. I think it is roughly the same length and complexity in the end.

Anonymous · 7 December 2023

That’s fascinating Terry. What do you see as the value of golfing the code? As someone with a background in commercial programming, I’m used to trying to make code as clear and simple as possible. To my eyes, the golfed code is significantly harder to read. But I’m not a mathematician, so I may well lack the right perspective here.



Terence Tao · 7 December 2023

From the mathematical side, I think it comes down to the principle of “giving appropriate amounts of detail”, which I write about here. Ideally, each step in a proof should be of the natural size that a mathematician would interpret as a significant step towards resolving the problem, and all steps should be broadly of similar size. The initial blueprint, breaking up the proof into six very tiny steps, satisfies the second property but not the first in my opinion, whereas the golfed version containing three steps is a bit more consonant with the high-level description of the proof rather than the low-level one.
For large projects (and particularly for contributing to Mathlib, the largest project of all), code is often “golf”ed not so much for brevity as for compilation speed. Powerful and intuitive tactics such as `simp` that perform an extensive search as part of their execution are often replaced with much more explicit `low level` tactics that are much faster but also harder to read. (There is a tactic called `simp?` which runs `simp` but then prints the specific simp lemmas used, which can then be used for this sort of compilation golf.) Perhaps this practice also influences the default aesthetic that the Lean formalization community has regarding how to write proofs, in that they tend to favor compactified arguments over long discursive ones in their Lean code. I like the Blueprint model in which LaTeX is used to describe the human readable version of the proof, while Lean focuses on its strength, which is to describe the machine readable version, but there are links between the two.



Anonymous · 21 June 2025

Hmmm I tend to disagree with the first argument regarding the step size that a mathematician would interpret as significant since different levels of education and even just different levels of expertise among specialists tend to have a relatively different needs of detail and what would imply “good size” here. From the point of view of automation I think the optimal way forward is to have Lean provide various levels of detail according to each reader’s needs – from overviews for someone who is experienced in the particular field to more granular approach for someone who is not. I guess the “?” versions are a step in this direction already and we are not quite there but at the very least here we can conclude that the “level of golfing” correlates with the reader’s experience in the given set of arguments.

A slightly longer Lean 4 proof tour – Cyber Geeks Global · 7 December 2023

[…] Comments…Read More […]



Richard · 7 December 2023

A thing I find off-putting about attempting to look at anything in Lean is the use of whacky 1970s-era computer language naming. Especially so with arbitrary and inconsistent abbreviations (abrvs, abbrevs, abs?) but also finSetSummand_unKnown_why_dothis.

“congr” WTF?

It’s especially difficult to navigate this salad of capitalization and arbitrary punctuation and runtgethrwds if one’s native language is not English. linear_arithmetic (better, linear-arithmetic, because normal English used by normal humans uses hypthens to join compound words) vs “linarith” or “linArith”, or God save us, “Linarith”, not only means a hell of a lot more to a casual reader, but makes it even possible for English non-adepts to *begin* to understand what’s going on, using the full words they already know because they’re actually in use in actual mathematics, aided by looking up normal words in normal dictionaries. (Yes, “linarith” is easy, but lots of other stuff, pretty much anything beyond this, really is not.)

I get that C programmers in 1970 were very concerned about every byte that the programs took up, and that naming somethign “inj” instead of “injective-functor” would save a lot of repetitive typing in the very primitive text editors of the time, but this hasn’t been the case for 40 years now … and yet computer-ese gibberish refuses to die.

I do understand that Lean itself wasn’t expressly designed for mathematics per se.

I get that it’s written by (and incredibly generously shared by) people completely steeped in decades of unpleasant computer language conventions. Do fish have a word for water?

I keep reading that Lean proofs “aren’t meant to be read” (because having computer programs, which is what Lean proofs are, be impenetrable, even at a superficial skimming level, is somehow a good thing.) But “proofs not mean to be read” are exactly what is exposed every time somebody new tries to even casually look at this formalization business.

In the end, there’s so much ancient computer programming language-ese exposed and inflicted; it’s off-putting. Trauma inflicted on each generation of computer programmers by the their preDecssrs is now inflicted more widely.

It’s truly nice that one can use finally use characters like “$\prec$” and “$\bigcap$” and “$\bigotimes$” in the future in 2023. But the “words” formed by run-together roman letters in between these familar, comprehensible operators? Oof.

Couldn’t Math_lib_hvDoneBetr? For mathematicians? It’s a tough tough slog.



Terence Tao · 7 December 2023

Mathlib’s naming conventions are laid out in this web page. One rationale for such conventions is to be able to quickly recognize whether a given term refers to a proposition, type, variable, or tactic (much as how in English, Capitalization is used to indicate Proper Nouns, ALLCAPS

is used to indicate acronyms and initialisms, periods are used to indicate abbreviations (e.g., abbrev., etc.), *italics* are used to denote foreign words that are used *verbatim*, and so forth).
Another key motivation for encouraging a rather formal naming system here is to make it easier to find specific Mathlib routines that achieve specific tasks. For instance, to find the lemma that establishes the telescoping series formula $\sum_{i=0}^{n} f(i) - f(i+1) = f(0) - f(n+1)$, I knew from the naming conventions that the formula name should probably involve `Finset`, `sum`, `sub`, and possibly also `range` or `eq`, and indeed by entering these terms into a Mathlib search engine one can readily locate `Finset.sum_range_sub`` which establishes precisely this fact. Naming this fact `telescoping_series_identity` or `Lemma 3.1.4.7` would make it more difficult to reliably locate the tool. (Though in a few years this issue may become moot, as AI-powered semantic search engines become more powerful, as well as AI-powered ways to convert Lean code into human-readable proofs.)
For personal Lean projects one can always introduce one's own personal abbreviations to replace the default names given by Mathlib of course, much as one can add a personal set of macros to replace LaTeX's default names for various mathematical symbols with names that one personally prefers.
Also, `congr` is an abbreviation of "congruence" (and `gcongr` is an abbreviation for "generalized congruence"). Hyphens can't be used in a mathematical language as they are already reserved for subtraction: `linear-arithmetic` would be interpreted as the difference of `linear` and `arithmetic`.



Anonymous · 7 December 2023

Thank you for making this post (all the screenshots and special Lean style must take a lot of time). One quick question: is Lean able to look at a proof, and say, oh this proposition always works for other fields (instead of reals). Well, not very useful in this case, but hypothetically. Humans can probably spot it easily, but I wonder if you've had the experience that you have missed a trivial extension which was "pointed out" by Lean. Thanks!



Terence Tao · 8 December 2023

Well, Lean's compiler has a "linter" that points out if a variable or hypothesis in the proposition was never actually used in the proof, so that is one simple way in which trivial extensions can be spotted. Another experience I have had is that sometimes, after a proposition has been proven, someone in the project realizes that in order to apply the proposition neatly in some other part of the project, one has to slightly modify a hypothesis in one of the proposition. Often one can just modify that hypothesis and watch what the compiler does. A surprisingly large fraction of the proof often still compiles under minor modifications, especially if the proof has not been prematurely optimized and one still uses broad tactics such as `simp` or `linarith` that don't require precisely specifying all the steps, but instead lets the compiler automatically locate the hypotheses, parameters, and methods needed to resolve the problem. Generally the compiler will only throw a handful of errors under a minor modification, which can then be dealt with by hand (and the process may recurse for a few iterations, because of steps after the first error that the initial compiler run did not even reach), but with well-designed proofs, the process of modifying a proposition is often substantially easier than writing a proof from scratch. (Indeed it is this aspect of formalization where I think we really have a chance in the future of being in a situation where the task is *faster* to perform in a formal framework than in traditional pen-and-paper (and LaTeX) framework.)



Anonymous · 8 December 2023

Does it?

Tony Carbery · 9 December 2023

At the risk of introducing a red herring to the conversation, I'd be interested if there were a Lean-type proof of the following free-standing inequality involving non-negative real numbers (or alternatively non-negative integers). We have three sequences a_j, b_j, c_j (indexed by non-negative integers j) and one sequence $K(i,j,k)$ (indexed by triples of non-negative integers). The inequality is
$\left(\sum K(i,j,k)a_i b_j c_k\right)^4 \leq A(\sum a_i^4)(\sum b_j^4)(\sum c_k^4)$
where the sums are over all relevant indices, and where the quantity A is given by
$\sum K(s_1,j_1,k_1)K(s_1,s_2,k_2)K(x_3,s_2,s_3)K(x_4,y_4,s_3)$
where the sum is over all visible indices. (The last expression is more readily visualised by writing the four terms in a vertical stack: we contract the indices 1 and 2, in the first column, indices 2 and 3 in the second column and indices 3 and 4 in the third column. This picture makes it apparent that the inequality in question is simply the case $n = 3$ of a more general family of inequalities, the case $n = 1$ of such is the Cauchy–Schwarz inequality.)
I realise this question looks really artificial and horribly technical, but I've not been able to find a simpler model which raises the same issues.

> **Terence Tao** · 9 December 2023
>
> This is an interesting inequality – a variant of the Blakely-Roy inequality? At its current stage of development, Lean isn't really capable of coming up with proofs on its own unless there is already a human-readable proof available already (or if a *very* short proof is available). Do you have such a human-readable proof? If so, I can take a stab at trying to formalize it. I'm guessing that the tensor power trick argument that Nets and I used to prove Lemma 2.1 of https://arxiv.org/pdf/math/9906097.pdf may give a proof that is not too difficult to formalize in Lean (which has no essential difficulty dealing with real analysis techniques such as the passage to a limit arising in the tensor power trick), but first (as in this post) one would have to write a human-readable proof first.

Anonymous · 10 December 2023

Yes, indeed, this inequality was inspired by your paper with Nets that you mention, and can indeed be proved by the tensor power trick. I was always intrigued that I couldn't prove it "by hand" even when restricting the inputs to range over finite collections — even in this finitistic setting a limiting argument seemed to be needed. (I was hoping that a computer would one day prove it for me without recourse to limiting arguments, and, if I'm honest, that was my real motivation in asking the question here.) The human-readable proof is available at \url{https://www.ams.org/journals/proc/2004-132-11/S0002-9939-04-07565-3/?active=current}. Thanks for the heads up on Blakely-Roy, which I was sadly unaware of.

> **Terence Tao** · 10 December 2023
>
> Ah, I remember this paper now. One could translate that paper step by step into Lean, but it would be rather time-consuming and not all that edifying. Lean is a formal proof assistant rather than an automatic theorem prover; it can formalize existing proofs, but at the current state of development it is not all that useful in discovering new proofs.
> On the other hand, I believe one can interpret your proof through the lens of entropy inequalities, and in particular through the Legendre-Fenchel formula
> $\log \sum_{x \in S} e^{f(x)} = \sup_X \mathbf{E} f(X) + \mathbf{H}[X]$
> for any finite set S and any function $f : S \to \mathbf{R}$, where the supremum is over all S-valued random variables and $\mathbf{H}[X] = \sum_{s \in S} \mathbf{P}[X = s] \log \frac{1}{\mathbf{P}[X=s]}$ is the Shannon entropy of X. (This formula is perhaps more familiar to you as the duality relation between the Orlicz spaces e^L

and $L \log L$.) It is then possible to prove your inequality from the Shannon entropy inequalities, without explicit use of the tensor power trick. I'll write this up in a separate blog post.

 **YahyaAA1** · 15 December 2023

Terry, it's instructive to see how you're evolving better strategies for working with this new(ish) proof assistant tool:
'I am finding the pre-planning step of the proof (using an informal "blueprint" to break the proof down into extremely granular pieces) to make the formalization process significantly easier than in the past (when I often adopted a sequential process of writing one line of code at a time without first sketching out a skeleton of the argument).'
You talk of using LEAN's "tactics". Do you think it will soon also have "strategies", at a higher hierarchical level, perhaps as "macros" of its tactics? And that would let us "blueprint" the proof at such a level?

 **Terence Tao** · 16 December 2023

Certainly there is work in this direction, particularly using AI tools either in a "bottom up" fashion to develop smarter "meta-tactics" that can solve in one shot problems that currently require several lines of Lean code, or in a "top down" fashion to try to break up a long argument into smaller chunks that could then be converted to a blueprint type format. See for instance "Sagredo" or "LeanDojo" for some current experiments in the former direction. There is also the recent "FunSearch" proof of concept which gained some attention in the popular media, which perhaps is closer to the "automatic strategy generation" approach you were suggesting.

 **Anonymous** · 1 January 2024

How is the set of natural numbers defined in Lean4?

 **Anonymous** · 1 January 2024

For all $x \in N$,
$(x+1)^2 = x^2 + 2x + 1$
Also, for all $x \in Z$, $(x+1)^2 = x^2 + 2x + 1$
Are both provable in Lean 3 & 4?

 **Terence Tao** · 2 January 2024

Sure, they are both easily provable by Lean's `ring` tactic, see here. The natural numbers are defined in Lean as an inductive type `Nat` (or ℕ): roughly speaking, a natural number is either zero, or a successor of a natural number, and these are the only ways to produce natural numbers. (This is slightly different from the orthodox ZFC construction of the natural numbers, but ends up being functionally equivalent in practice.)

 **Anonymous** · 3 January 2024

Many thanks for a crystal clear answer, more on the type theoretic foundation later. We are just now at the beginning of 2024.
For large mathematical objects, we have formalized so many kinds of objects in various proof assistants. In forseeable years, it seems that Coq, Lean, and Metamath will be *standard* among formalized mathematics.
$\forall n \in \mathrm{N}. \sum_{i=0}^{n} \binom{n}{k} = 2^n$
How would we formalize the combinatorial proof of the above statement? Similarly, large formalization projects are struggling to do what everyday mathematicians can easily do: arithmetic topology.



Terence Tao · 21 January 2024

The combinatorial statement – that the powerset of a set of cardinality n decomposes as the disjoint union of the i-element subsets for $i = 0, \ldots, k$ – is formalized in Lean here, and it is not difficult to then conclude the binomial identity: see here. (This identity is also implemented here.)



Anonymous · 1 January 2024

N is not a type.



Anonymous · 27 January 2024

The Goldbach conjecture is true becase the fractional remainders after deductions of 2/P where P= prime less than sqrt:x always end leaving may primes, short, simple absolute with a clear example demonstrating just how impossible it is not to be true.

Every even number x is always the sum of 2 prime numbers because with every increase in value of x (always 2 integers more than the last) then all odd numbers below x/2 move one further away from x/2 and all above x/2 move one closer, so the odd numbers always pair with another odd number. So if one odd number a distance k below x/2 is a multiple of a Prime (Pn) then we can rule out it and the number a distance k above x/2 as being a prime pair. So by eliminating all multiples of P<√x we can figure out how many primes will be left over and these must pair, add together to equal x. We do this by dividing x by 2 to get the number of odd numbers below x then subtract 2 by all multiples of primes <√x which is any remaining number divided by 2/P where P is the next higher prime eg:

There are always more primes left over below and above x/2 after such pairings have been eliminated (as demonstrated in this example below where x=10,004 which is illustrative for all values of x) so those primes remaining must be prime pairs. So the Goldbach conjecture is definitely true.

To demonstrate that with an example let's look at a number with no prime factors to get the least possible number of possible prime pairs

X=10,004/2=5000
5000×2/3=333.33333
333.33333×2/5=2,666.6666
2,666.6666×2/7=1904.76184
1904.76184×2/11=346.32033 =
1,558.44151-2/13=1318.6812776923 1,318.6812776923'-2/17=155.13897
1,163.542303846-2/19=1,041.0641665991
1,041.0641665991-2/23=950.5368477644
950.5368477644-2/29=884.9825824058
884.9825824058-2/31=827.8869319542
827.8869319542-2/37=783.1362870353
783.1362870353-2/41=744.934516936
744.934516936-2/43=710.2863998692
710.2863998692-2/47=680.0614466833
680.0614466833-2/53=654.3998826575
654.3998826575-2/59=632.2168357878
632.2168357878-2/61=611.4884149423
611.4884149423-2/67=593.2350294216
593.2350294216-2/71=576.5241835224
576.5241835224-2/73=560.7290004122
560.7290004122-2/79=546.5333295157
546.5333295157-2/83=533.363851696
533.363851696-2/89=521.3781471635

521.3781471635-2/97=510.6280822735
That's all primes <√x
Even if we were to include multiples of primes greater than <√x and even as the values of x go towards gazillions of gazillions of bazillions and beyond the figure will eventually converge to a percentage of x much higher than 2 for one Prime pair as the trend below demonstrates, note, primes above <√x are not relevant, I just include these to emphasise how impossible it is to not have prime pairs=x and the higher the value of x the greater the number of prime pairs will be left
510.6280822735-2/101=500.5166350998
500.5166350998-2/103=490.7978654862
490.7978654862-2/107=481.624073608
481.624073608-2/109=472.7869346427
472.7869346427-2/113=464.419024295
464.419024295-2/127=457.105338873
457.105338873-2/131=450.1266314093
450.1266314093-2/137=443.5554397099
443.5554397099-2/139=437.1733470522
437.1733470522-2/149=431.305248434
431.305248434-2/151…..etc
This and my proof to the Collatz conjecture with voiceover for visually impaired are in a mathematical proof playlist on my youtube channel Sean A Gilligan maths & physics.

lucyeinsteinbfca8f877e · 27 January 2024

Made an error in the previous comment and dont know how to delelete it sorry, of topic but homeless and unable to access the peer review system so puttng it here for obvious reasons. Thanks Terry Fatigue is a killer! I made one error in this proof and it messed up all further numbers, here it is fixed. The proof to the Goldbach conjecture. Please take it with the level of seriousness it warrants and share.
The Goldbach conjecture is true, every even number x is always the sum of 2 prime numbers because with every increase in value of x (always 2 integers more than the last) then all odd numbers below x/2 move one further away from x/2 and all above x/2 move one closer, so the odd numbers always pair with another odd number. So if one odd number a distance k below x/2 is a multiple of a Prime (Pn) then we can rule out it and the number a distance k above x/2 as being a prime pair. So by eliminating all multiples of P<√x we can figure out how many primes will be left over and these must pair, add together to equal x. We do this by dividing x by 2 to get the number of odd numbers below x then subtract 2 by all multiples of primes <√x which is any remaining number divided by 2/P where P is the next higher prime eg:
There are always more primes left over below and above x/2 after such pairings have been eliminated (as demonstrated in this example below where x=10,004 which is illustrative for all values of x) so those primes remaining must be prime pairs. So the Goldbach conjecture is definitely true.
To demonstrate that with an example let's look at a number with no prime factors to get the least possible number of possible prime pairs
X=10,004/2=5002
5002-2/3=5,002−((5,002)×(2/3)=
1,667.3333333333-2/5=1000.4
1000.4-2/7=714.5714285714
714.5714285714-2/11=584.6493506493
584.6493506493-2/13=494.7032967033
494.7032967033-2/17=436.5029088559
436.5029088559-2/19=390.5552342395

390.5552342395-2/23=356.593909523
356.593909523-2/29=332.0012261076
332.0012261076-2/31=310.5817921652
310.5817921652-2/37=293.7935871833
293.7935871833-2/41=279.4621926866
279.4621926866-2/43=266.4639511663
266.4639511663-2/47=255.1250596273
255.1250596273-2/53=245.4976988866
245.4976988866-2/59=237.1757429921
237.1757429921-2/61=229.3994891235
229.3994891235-2/67=222.5517431795
222.5517431795-2/71=216.2826799913
216.2826799913-2/73=210.3571271148
210.3571271148-2/79=205.0316302258
205.0316302258-2/83=200.0911090155
200.0911090155-2/89=195.5946795994
195.5946795994-2/97=191.5617996077
That's less all multiples of primes <√x where x=10,004 not even allowing for some odds which are not primes to pair up, which they will and still we get a MINIMUM of around 95 prime pairs adding to x
Even if we were to include multiples of primes greater than <√x and even as the values of x go towards gazillions of gazillions of bazillions and beyond the figure will eventually converge to a percentage of x much higher than 2 for one Prime pair which further emphasises just how impossible it is to not have prime pairs adding to x.
This and my proof to the Collatz conjecture are also in short video format usually about 3 to 5 minutes, with voiceover for visually impaired in a mathematical proof playlist on my youtube channel Sean A Gilligan maths & physics.



Anonymous · 29 January 2024

Claim: Every infinite computably enumerable set has an infinite computable subset.
Proof: Consider a computably enumerable set $A \subseteq \Sigma^*$ where $\Sigma = \{0, 1\}$. Define the following order in Σ^*: for two strings $a, b \in \Sigma^*$, we say $a < b$ if either $|a| < |b|$ (where $|x|$ denotes the length of the string x), or $|a| = |b|$ and $a \underset{\text{lex}}{<} b$ (i.e. a precedes b in the usual lexicographic order).
So some computable enumerator of A enumerates A as a list: $a_0, a_1, ...$ Call an a_i a *pioneer* if either $i = 0$ (well, the very first one is always a pioneer), or $i > 0$ and $\forall j < i (a_j < a_i)$ (a_i beats all the so-far-enumerated ones). Let $S = \{a_i \in A \mid a_i \text{ is a pioneer}\}$. Clearly, $S \subseteq A$.
S is computable by the following machine. On an input string $x \in \Sigma^*$, this machine just runs the enumerator of A. Each time the enumerator appends a string a_i for $i >= 0$ to the list of so-far-printed-out strings, our machine *pauses* the enumerator to check if the last-appended string is a *pioneer* or not. If a_i is not a pioneer, our machine resumes the enumerator. If a_i is a *pioneer*, our machine compares x and a_i: if $x = a_i$, **Accept** and halt; otherwise, if $x > a_i$ (so there's still hope) resume the enumerator, if not, i.e. $x < a_i$, well the latest pioneer just goes further than x, hopeless, **Reject** and halt.
Note that the subsequence of pioneers is strictly increasing. Eventually, the latest pioneer must be greater than x. So, x is accepted if and only if it is a pioneer at some time in the timeline. For us, S is computable.
Strictly mathematically put, we should call an index i a pioneer index instead of a_i, but for an easy-to-understand manner, we did it as above.

Anonymous · 16 February 2024

Hey Tao, just wanted to say your post has inspired me to learn Lean and I am loving it. Thanks!

Theorem Proving using Machine Learning – Lean 4 – A Guide To Mathematical Studies · 15 April 2024

[…] Formalizing the proof of PFR in Lean4 using Blueprint: a short tour A slightly longer Lean 4 proof tour […]

How the Lean language brings math to coding and coding to math - Rockstary Reviews · 16 August 2024

[…] feature for both the mathematical and software verification communities because it eliminates the trust bottleneck. It doesn't matter who you are; if Lean checked your proof, the whole world can build on top […]

Anonymous · 19 August 2024

https://pubs.rsc.org/en/content/articlelanding/2024/dd/d3dd00077j

Formalizing chemical physics using the Lean theorem prover

How the Lean language brings math to coding and coding to math | My Blog · 1 September 2024

[…] feature for both the mathematical and software verification communities because it eliminates the trust bottleneck. It doesn't matter who you are; if Lean checked your proof, the whole world can build on top […]

Anonymous · 9 September 2024

Hi Terry,

Since I read this post, I have been meaning to ask you this: do you have any experience with the Isabelle/HOL theorem prover?

I ask because when I read your closing remarks about how AI may one day be able to close these small gaps in formal proofs, I thought to myself that Isabelle/HOL's existing Sledgehammer tool should easily be able to tackle these gaps. And indeed, I recreated this problem in Isabelle/HOL, and found that Sledgehammer had no trouble automatically proving every step in the chain of inequalities that you laid out.

If you are interested in the current state of automatic formalization, you should play around with Isabelle/HOL and try out its Sledgehammer tool on problems like this.

While I'm sure Lean's automation will catch up to Sledgehammer, I believe Sledgehammer is still the best automation tool currently available.

> **Anonymous** · 9 September 2024
>
> I also meant to add: I have really been enjoying reading about your experiences in Lean, and I would love to hear your thoughts on how various theorem provers compare (e.g. Isabelle/HOL vs. Lean).
>
> (P.S., I may have accidentally double-posted my original comment; sorry).

masharpe · 19 October 2024

Thanks, this was very helpful to me for learning idiomatic usage of mathlib4.

Another point where I'd be curious to learn from expert experience is regarding how to formulate definitions well. There seem to be subtly different options for how to do it, and the choice impacts all statements and proofs that depend on the definition, so it seems important to get right.

> **masharpe** · 26 October 2024
>
> I also found it interesting that you reformulated the proof as an "informal blueprint" first before attempting the full formalization. It's similar the use of pseudocode for programming, in that

it's more explicit than needed for communicating the key ideas to a human yet omits details of the target programming environment (here, Lean and mathlib). I'd imagine that the same pseudoformalization might serve as a suitable starting point for a formalization in most other formal proof systems, too.

Anonymous · 23 August 2026

"gcongr with i hi" doesn't work anymore. I get the error "Unknown identifier `i`" .
I use Lean version 4.32.1 .

Terence Tao · 23 August 2026

Lean's library of tactics, syntax, and methods have undergone various tweaks in the last three years. There is insufficient information provided to diagnose your specific problem, but you can try pasting your code into a modern LLM who may be able to suggest the new syntax that typechecks. (I find using a coding agent integrated into one's IDE to be particularly convenient for such a task as it can run the typechecker automatically, but even a chatbot interface should be able to carry out this task with good accuracy nowadays.)

powerful020b56fc4e · 25 August 2026

This code works :

```
import Mathlib.Data.Real.Basic
import Mathlib.Data.Finset.Basic
import Mathlib.Algebra.BigOperators.Intervals
import Mathlib.Algebra.Order.BigOperators.Group.Finset
import Mathlib.Tactic.FieldSimp
import Mathlib.Tactic.Linarith
import Mathlib.Tactic.GCongr
open Finset BigOperators
example (a D : ℕ → ℝ) (hD : ∀ k, a k ≤ D k − D (k + 1)) (hpos : ∀ k, 0 ≤ D k)
(ha : Antitone a) (k : ℕ) : a k ≤ D 0 / (k + 1) :=
calc
a k = (Σ _i ∈ range (k+1), a k) / (k+1) := by simp ; field_simp
_ ≤ (Σ i ∈ range (k+1), (D i − D (i+1))) / (k+1) := by
apply div_le_div_of_nonneg_right _ (by positivity)
exact sum_le_sum fun i hi => (ha (by simpa using hi)).trans (hD i)
_ ≤ D 0 / (k+1) := by gcongr; rw [sum_range_sub']; linarith [hpos (k+1)]
```